



TensorBoard

TensorFlow's visualization toolkit

A web application for inspecting the model training

1/2569

Rattachai Wongtanawijit

241-353 Artificial Intelligence Ecosystem Module

tensorflow.org/tensorboard

Department of Computer Engineering, Prince of Songkla University 2024

Chapter Overview & Objectives

1. Google TensorBoard

Neural Network Visualization: TensorBoard: *Directed Acyclic Layer Graph*
Model Training Log with TensorBoard

2. YOLO Loss Functions

Branch Losses {*object-ness, class, box regression*}

3. Non-constant Learning Rates: Learning Rate Scheduler

4. Model Weight Regularization

5. Team Assignments:

Chapter Overview & Objectives

5. Team Assignments: using custom dataset to ...

- **Part 1:** Transfer Learning **YOLOv5 Box detector**
- **Part 2:** Transfer Learning **YOLOv5 Segmentation**
- **Part 3:** Training a custom **U-Net from scratch**

What is TensorBoard?

A real-time interactive logger in a **web-application dashboard**

- Tracking and visualizing metrics
 - Visualizing the model graph
 - Histograms of weights, biases, or other tensors over time
 - Projecting embeddings to a lower dimensional space
 - Displaying images, text, and audio data
-
- Run & access on local machine with web browser through `https://localhost:6006`
 - Ultralytics YOLO Integration
<https://www.ultralytics.com/blog/visualizing-training-metrics-with-the-tensorboard-integration>

Starting TensorBoard

https://www.tensorflow.org/tensorboard/get_started

```
$pip install tensorboard
```

```
$tensorboard --logdir="runs"
```

“run” is the directory of the logging .protobuf files

Then go to <https://localhost:6006>

TensorBoard with PyTorch Tutorial

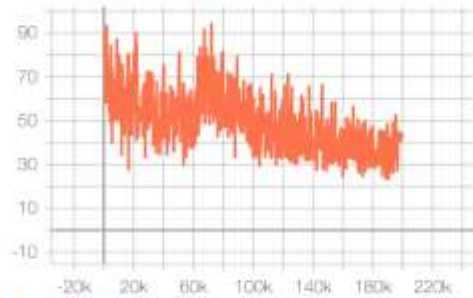
https://pytorch.org/tutorials/recipes/recipes/tensorboard_with_pytorch.html

Training metrics Visualization

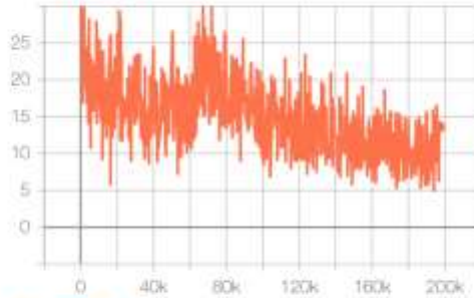
Loss

6

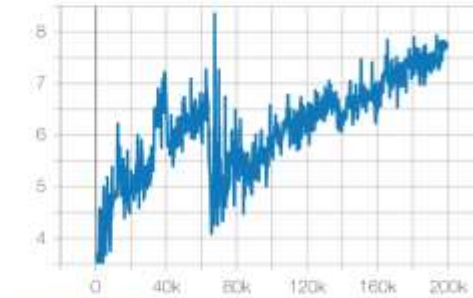
HardExampleMiner/NumNegatives
tag: Loss/HardExampleMiner/NumNegatives



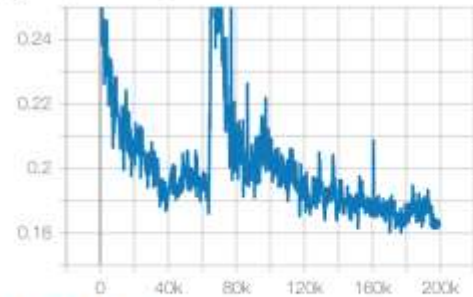
HardExampleMiner/NumPositives
tag: Loss/HardExampleMiner/NumPositives



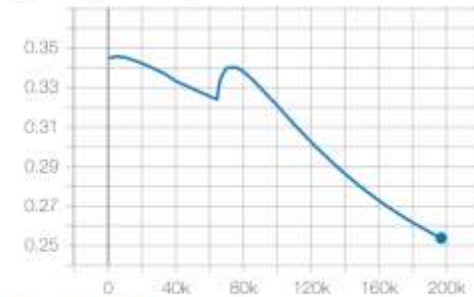
classification_loss
tag: Loss/classification_loss



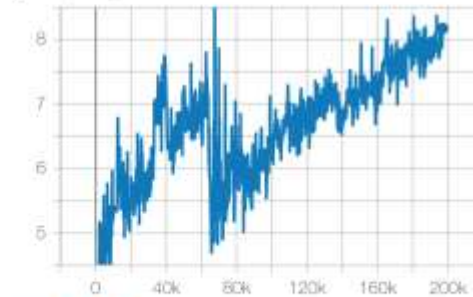
localization_loss
tag: Loss/localization_loss



regularization_loss
tag: Loss/regularization_loss



total_loss
tag: Loss/total_loss



- Logger implemented using Google Protobuf
- Plot panes called tags or cards

Training Terminology

Stochastic Gradient Descent (SGD)

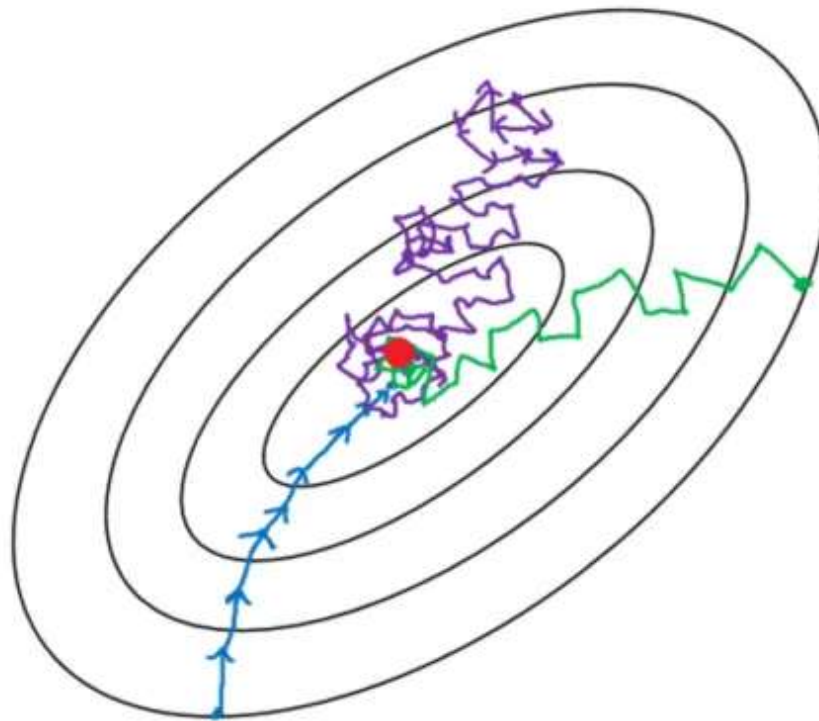
Mini-batch Gradient Descent

Batch size

Iteration

Epoch

Loss function (objective function)



- Batch gradient descent
- Mini-batch gradient Descent
- Stochastic gradient descent

Stochastic Gradient Descent : SGD

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla \mathcal{L}(\mathbf{w}_t; \mathbf{x}_i, y_i)$$

- Aka. Batch size = 1
- High gradient noise, volatile convergence trajectory
- **Low Hardware Efficiency**: may under-utilizes GPU parallel compute capabilities

Mini-batch Gradient Descent

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \left(\frac{1}{B} \sum_{i=1}^B \nabla \mathcal{L}(\mathbf{w}_t; \mathbf{x}_i, y_i) \right)$$

- Batch size > 1
- Averaged gradients produce a smoother, faster convergence path
- High Hardware Efficiency (maximizes SIMD/GPU matrix multiplication throughput)

YOLO Loss (*object bounding box detection*)

Object-ness loss *measure is any object there?*

Class loss *measure which class that object belongs to?*

} Cross Entropy Loss

Box regression loss *predict the bounding box sizes and center location*

- Box's center – x-axis
 - Box's center – y-axis
- } RMSE, offset from anchor center, detection map resolution
- Box's width
 - Box's height
- } offset from anchors, RMSE

YOLO Loss (*object bounding box detection*)

1. Object-ness loss

2. Class loss

3. Box regression loss

- Box's center – x-axis
- Box's center – y-axis
- Box's width
- Box's height



Sum to **Total_loss**, use in back-prop weights adjustment relates to gradients

Regularization loss

Assumption: *Network that converges properly (without in-layer weight bias), the weights in kernels should normally distributed, centered at zero or nearly zero*

- So, if the **RMS (root mean square)** of weights in each layers being high (far away from zero), the network should be penalized.
- We directly add the term **reg_loss** to the **total_loss**
- **RMSE called L2-Regularization**

Regularization loss

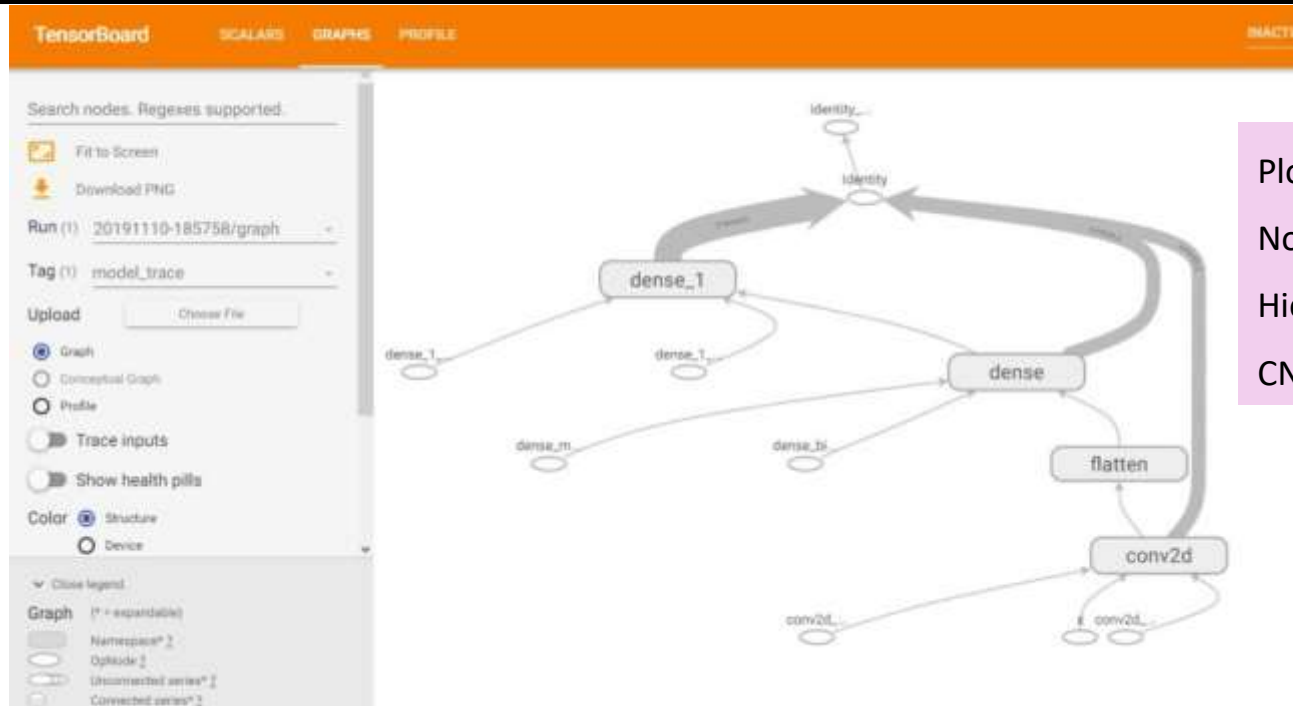
The standard **data loss** (such as Cross-Entropy or Mean Squared Error) measures how the model fits the training data,

The **regularization loss** measures how complex or extreme the model parameters (weights) are.

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}}(\mathbf{y}, \hat{\mathbf{y}}) + \lambda \Omega(\mathbf{W})$$

- Penalty term added to Objective Function to avoid Model Complexity
- **Prevent** Overfitting, Generalization

Model layers (Ops) in graph



Plot the NN in a computation graph.
Node represents tensor operation.
Hierarchical node levels = NN layers.
CNN = Directed-acyclic graph.

Tutorials:

<https://www.analyticsvidhya.com/blog/2021/04/a-complete-guide-to-tensorboard/>

<https://www.tensorflow.org/tensorboard/graphs>

Examples: TensorBoard API with PyTorch

Example for logging scalar measurements :

- [Github.com/pytorch/tutorials/blob/main/recipes_source/recipes/tensorboard_with_pytorch.py](https://github.com/pytorch/tutorials/blob/main/recipes_source/recipes/tensorboard_with_pytorch.py)
- [Pytorch.org/tutorials/recipes/recipes/tensorboard_with_pytorch.html](https://pytorch.org/tutorials/recipes/recipes/tensorboard_with_pytorch.html)

Visualizing Models, Data, and Training with TensorBoard

[Pytorch.org/tutorials/intermediate/tensorboard_tutorial.html](https://pytorch.org/tutorials/intermediate/tensorboard_tutorial.html)

PyTorch documentation on TensorBoard

<https://pytorch.org/docs/stable/tensorboard.html>

Team Assignment – Part 1: YOLO-Box (1)

Starts with `$pip install tensorboard`

1. Train a YOLOv5 model using `train.py` to detect bboxes with your custom dataset.
 - *Split dataset into two sets: {train (100), test (100)}*
 - YOLOv5 code has already integrated loggers called “Comet ML” (not free www.comet.com) and “Clear ML” clear.ml for automated ML deployments. *We do not use that, turn off this feature.*
 - Use pretrained-YOLO pick one from n or s variants
 - You need adjust `train.py` to work with TensorBoard
 - You also need to adjust `data.yaml` to match with your datasets.

Team Assignment – Part 1 (2) YOLO-Box

2. Run model losses calculation for **at least 30 iterations**
(*adjusts epoch number and batch size depend on your resources*).
You can use your machine or computer in R302.
3. Plot these metrics in separate plot panes against iterations.
 1. Overall loss (mean or sum of loss, **total_loss**)
 2. Object-ness loss
 3. Classification loss
 4. Box regression loss
 5. Default scheduled learning rate (LR)

Team Assignment – Part 1 YOLO-Box (3)

Submission: **In a report**

1. Show all training loss plots in TensorBoard (*screenshot/image export*)
2. Show evaluation metrics plot in TensorBoard (*screenshot/image export*)
3. Plot the Learning Rate.
4. (option) Plot the weight distribution over time.
5. Show sample inference of 3 images from your dataset test split.
6. Answer: How do you notice the network converge?

Due date in Microsoft-Teams

Ultralytics YOLOv5

Using train.py with specify
command arguments

```
$python train.py
  --data coco128.yaml
  --epochs 10
  --weights model/yolov5n.pt
  --batch-size 64
  --img 320
  --noval
```

Example of console verbose in training

```
0
Epoch      GPU_mem    box_loss    obj_loss    cls_loss    Instances    Size
  0/9         1.85G      0.08769     0.04644     0.03777         742      320: 100%|██████████| 2/2 [00:02<00:00, 1.05s/it]

1
Epoch      GPU_mem    box_loss    obj_loss    cls_loss    Instances    Size
  1/9         1.94G      0.0839     0.04745     0.03907         838      320: 100%|██████████| 2/2 [00:00<00:00, 7.54it/s]
```

Team Assignment – Part 2 YOLO-Seg (1)

Repeat Part 1

Train a YOLOv5 model using `segment/segment.py` to do segmentation to your custom dataset.

.. Please split Dish-200 into two sets: {train (100), test (100)}

- Use pretrained-YOLO pick one from n, s variants
- You need adjust `segment.py` to work with TensorBoard
- You also need to adjust `data.yaml` to match your datasets.
- Plot loss functions

Team Assignment – Part 2 YOLO-Seg (2)

Submission: In a report

1. Show all training loss plots in TensorBoard (*screenshot/image export*)
2. Show evaluation metrics plot in TensorBoard (*screenshot/image export*)
3. Plot the Learning Rate.
4. Show sample inference of 3 images from your dataset test split.
5. Answer: What is the loss function used in segmentation? Is it different from Part 1?

Due date in Microsoft-Teams

Note: Team Assignment – custom dataset (assignment 8)

Submission: Upload your dataset into PSU storage.

In report: Write a dataset card. Describe your method. Show basic statistics (mean, histogram, distribution plots.)

Dataset card: <https://huggingface.co/docs/hub/en/datasets-cards>

Question?



Reference Summary

Ultralytics & TensorBoard	"Visualize Training Metrics with the TensorBoard Integration 🚀," Ultralytics Blog
PyTorch & TensorBoard	"TensorBoard with PyTorch — PyTorch Tutorials," pytorch.org
TensorBoard Guide	S. Arora, Analytics Vidhya
TensorBoard Graphs	"TensorBoard: Graphs," TensorFlow
Pytorch-UNet (GitHub)	milesial, GitHub repository
Hugging Face Dataset Cards	"Dataset Cards," Hugging Face Docs